Quantization: A Model Compression Method

Ruize Xu Group Seminar 2023.3.22

1 Motivation

Quantization is a classical technique from signal processing that allows data compression. A quantized model executes some or all of the operations on tensors with reduced precision at lower bit widths rather than full precision (floating point) values. With the significant progress of large models, quantization is particularly relevant when memory and/or computational resources are severely restricted during training or inference. Therefore, this note is a quick glance at common quantization methods for neural networks, including principles and implementation.

2 Basic Concept

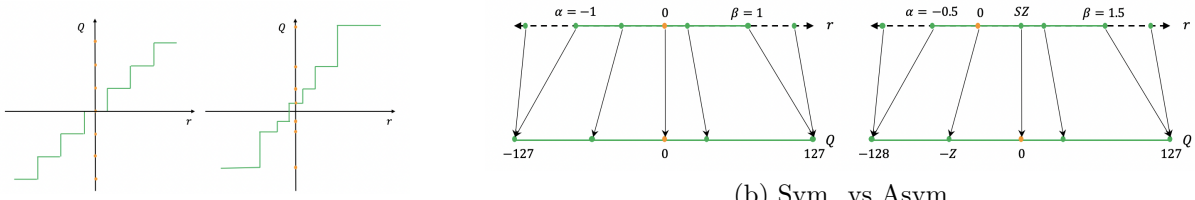
Typical quantization maps real values in the continuous domain into discrete lower-precision values [1]. A popular choice quantization function, *i.e. uniform quantization* is as follows:

$$Q(r) = Int(r/s) - Z, \tag{1}$$

where Q is the quantization operator, r is a real-valued input (activation or weight), S is a real-valued scaling factor, and Z is an integer zero point. Furthermore, the Int function maps a real value to an integer value through a rounding operation (e.g., round to nearest and truncation). In essence, this function is a mapping from real values r to some integer values.

3 Variants

3.1 Uniform vs Non-uniform



(a) Uni. vs Non-uni. (b) Sym. vs Asym.

Figure 1: (a) The distances between the quantized values are the same in uniform quantization, whereas they can vary in non-uniform quantization. (b) Symmetric quantization with restricted range maps real values to [-127, 127], and full range maps to [-128, 127] for 8-bit quantization.

3.2 Symmetric vs Asymmetric

This scaling factor S essentially divides a given range of real values r into a number of partitions:

$$S = \frac{\beta - \alpha}{2^b - 1}, \tag{2}$$

where $[\alpha, \beta]$ denotes the clipping range and b is the quantization bit width. The process of choosing the clipping range is often referred to as *calibration*. A straightforward choice is to use the min/max of the signal for the clipping range, i.e., $\alpha = r_{min}$, and $\beta = r_{max}$. Asymmetric quantization is especially important when the target weights or activations are imbalanced, e.g., ReLU always has non-negative values. Instead of the largest/smallest value, the i-th largest/smallest values are commonly used as β/α to avoid outliers.

3.3 Range Calibration: Static vs Dynamic

The clipping range of $[\alpha, \beta]$ can be determined in different ways. In *dynamic quantization*, this range is dynamically calculated for each activation map during runtime, requiring the real-time computation of the signal statistics (min, max, percentile, etc.) and often resulting in higher accuracy. In *Static quantization*, the clipping range is pre-calculated and static during inference. Multiple metrics are used in the pre-calculation to find the best range, including MSE, entropy, etc. Other work jointly optimizes the range variables and the weights in NN during training.

3.4 Granularity

The convolutional filters can have a different range of values. As such, one differentiator for quantization methods is the granularity of how the clipping range is calculated for the weights. **Layerwise:** considering all of the weights in convolutional filters of a layer; **Groupwise:** grouping multiple different channels inside a layer to calculate the clipping range; **Channelwise:** Using a fixed value for each convolutional filter independent of other channels.

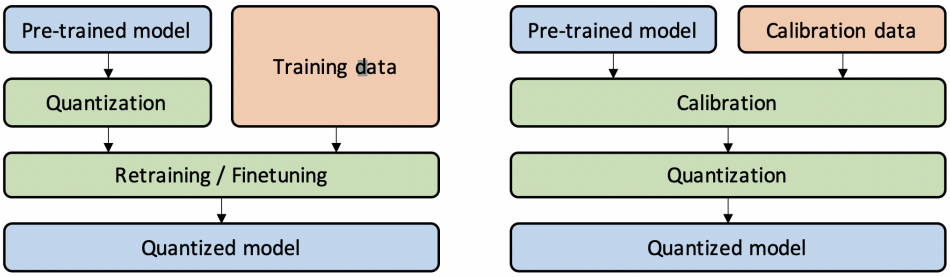


Figure 2: (left) Quantization-Aware Training for retraining/finetuning. (right) Post-Training Quantization for calibrating the pre-trained model.

3.5 Pipeline of application

Quantization Aware Training (QAT): retraining the pre-trained model with quantized parameters so that the model can converge to a point with better loss. To deal with the non-differentiable quantization, one way is to perform the backward pass with floating points, as quantized gradients result in high error. A practical way is *Straight Through Estimator (STE)*, which ignores the rounding operation and approximates it with an identity function. **Post-Training Quantization (PTQ)**: Performing the quantization and the adjustments of the weights without any fine-tuning. It has an additional advantage that it can be applied in situations where data is limited or unlabeled.

3.6 Advances

- Alternatives for **STE** for better backpropagation: stochastic quantization.
- Below 8 bits: optimization of operators and storage or mixed-precision quantization. An extreme example is binarization.
- Hardware-aware quantization: adapted for specific architectures.
- Vector Quantization: representing a real-number vector by $\mathbf{r} = \sum_{i=1}^{m} \alpha_i \mathbf{b}_i$, with the scaling factors $\alpha \in R$ and $\mathbf{b}_i \in \{-1, +1\}^n$. A large number of explorations exist(VQ-VAE, VQ-GAN, VQ-wav2vec, VQ-GPT, VQ-Diffusion, etc.).

4 Implementation

Pytorch official implementation of quantization, including *Dynamic Quantization, Static Quantization, QAT* and *PTQ*. `https://pytorch.org/docs/stable/quantization.html`

```
import torch
# create a floating model instance
model_fp32 = M()
# create a quantized model instance
model_int8 = torch.ao.quantization.quantize_dynamic(
    model_fp32,  # the original model
    {torch.nn.Linear},  # a set of layers to dynamically quantize
    dtype=torch.qint8)  # the target dtype for quantized weights
# run the model
input_fp32 = torch.randn(4, 4, 4, 4)
res = model_int8(input_fp32)
```

References

[1] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," *arXiv preprint arXiv:2103.13630*, 2021.